

Personal Finance Management System

Database Management Systems – Final Project

Nguyen Phuong Ngan
Student ID: 11245914 Class: DSEB 66B

National Economics University
Instructor: PhD. Tran Hung

May 2026

Agenda

- 1 Problem & Objectives
- 2 System Architecture
- 3 Database Design
- 4 Advanced Database Objects
- 5 Application Implementation
- 6 Conclusion

Problem Statement & Objectives

The Problem:

- Cash-based transactions in Vietnam make tracking finances difficult.
- Lack of structured budgeting leads to missed thresholds and untracked spending patterns.

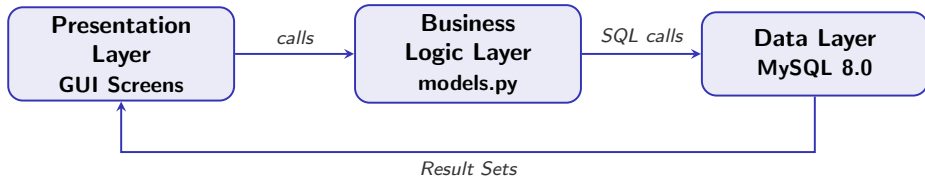
Project Objectives:

- Build a persistent, structured relational database using **MySQL** to enforce business rules.
- Deliver an intuitive **Python-based GUI** (CustomTkinter) for non-technical users.
- Automate data integrity (e.g., account balances) via **Full Lifecycle Triggers**.
- Provide actionable visual analytics (income vs. expense, category spending).

3-Layer System Architecture

The system strictly separates concerns to prevent SQL injection and improve maintainability:

- **Presentation Layer:** `main.py` (CustomTkinter). No SQL strings exist here.
- **Business Logic Layer:** `models.py` + `reports.py`. Handles parameters and connections.
- **Data Layer:** **MySQL 8.0**. Executes complex queries and maintains integrity.



Core Entities:

- Users, BankAccounts
- Income, Expenses, Categories
- Budgets (Monthly Limits)

Security & Access Control:

- **Restricted User:** `finance_app`
- **Least Privilege:** No DROP allowed; DELETE restricted to transactional data only.
- **Integrity:** Balances protected from manual UPDATE (Triggers only).

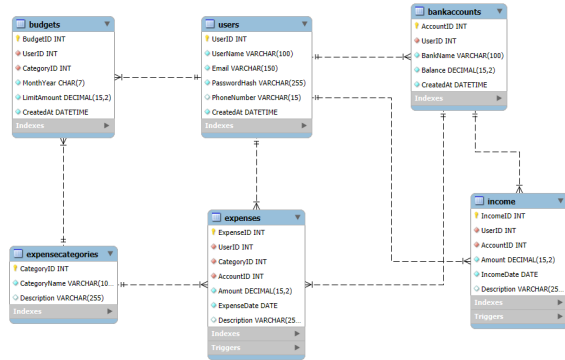


Figure: Relational Schema (MySQL)

Advanced Database Objects

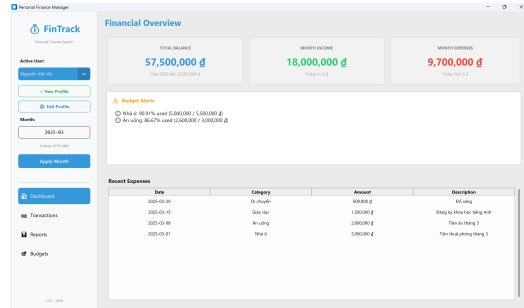
Business logic is pushed down to the Database Layer for absolute data integrity:

- **Full Lifecycle Triggers:** AFTER INSERT, UPDATE, and DELETE on Income/Expenses.
 - Automatically restores/adjusts balances even when moving funds between accounts.
- **Optimized Views:** vw_budget_status abstracts complex JOINS and computes usage percentages (OK / WARNING / EXCEEDED).
- **Stored Procedures & UDFs:** sp_monthly_report utilizes fn_total_expenses for logic reuse.
- **Indexes:** Composite B-tree indexes on (UserID, Date) for $O(\log n)$ retrieval.

Python GUI & Application Screens

Built with **CustomTkinter** and **Matplotlib**, featuring 4 main modules:

- **Dashboard:** Real-time KPI cards and budget alerts.
- **Transactions (CRUD):** Modal Popups for Edit/Delete with DB synchronization.
- **Budgets:** Interactive table with **In-line Deletion (Trash Icon)**.
- **Reports:** Visualized charts (Bar, Pie, Line trends) with instant refresh logic.



Key Achievements:

- Delivered a fully functional tracking system meeting NEU requirements.
- Successfully enforced business invariants reliably using MySQL Triggers.
- Implemented a clean, secure 3-layer architecture protecting the DB.

Future Improvements:

- Integration with Exchange Rate APIs for multi-currency.
- Predictive analytics for budget overrun warnings.
- Automated PDF report exports.

Thank you for your attention!

Demonstration & Q&A